电商销量预测系统 · 部署运维手册

版本 V1.0 | 2026-10-13 | 适用范围：单机部署（Linux / Windows 均可）

本文档是《项目任务书 V1.0.0》§5.2 要求的「部署运维手册」交付物，对应用户需求 `FR-SYS-03`。

与本文档配套的文档：《开发技术文档 V1.0》（口径与实现细节）、《测试报告 V1.0》（验收证据）、《API 接口文档 V1.0》（接口契约）、《项目说明 V1.0》（参数总表）。

目 录

0. 文档定位与阅读路径 4

1. 系统概览 4

1.1 架构 4

1.2 组件与端口 5

1.3 技术栈与版本（🔒 版本锁定，NFR-COMP-07） 5

1.4 运行期硬约束（写进验收红线） 6

2. 环境准备 7

2.1 硬件最低配置 7

2.2 系统依赖 7

2.3 Python 依赖安装（⚠ torch 必须单独装） 8

2.4 必需的文件资产（🔒 缺一不可） 8

2.5 环境变量（.env） 9

3. 部署 10

3.1 目录结构 10

3.2 首次部署（开发/联调态，3 条命令） 11
3.3 接口文档（/docs、/redoc）—— 🔒 资源已离线化 11
3.4 生产构建与 Nginx 12
3.5 部署后自检清单（🔒 逐项确认） 14
4. 配置详解 15
4.1 数据目录与 TTL 策略（三档） 15
4.2 日志 15
4.3 数据库参数（app/core/db.py，🔒 每次连接执行） 16
4.4 安全配置（生产必查） 16
5. 模型运维 17
5.1 模型资产 17
5.2 重训流程（🔒 GIT-7 纪律，顺序不可颠倒） 17
5.3 模型切换（管理端） 18
6. 服务运行 19
6.1 开发态前台运行 19
6.2 生产态 systemd（推荐） 19
6.3 启动自检链（出问题优先看这里） 20
7. 数据备份与恢复 20
7.1 备份（🔒 SQLite 必须用 .backup，不可直接 cp） 20
7.2 恢复 22
7.3 数据库迁移 22
8. 定期运维任务 23
9. 质量保障与回归 23
9.1 校验脚本矩阵（🔒 全部为可复现断言，非人工目视） 23
9.2 回归执行纪律（⚠ 血泪教训） 24
9.3 发布前必跑 25
10. 监控与容量 25

10.1 关键指标（⏱ 本机实测基线，2026-10-12） 25
10.2 容量估算 26
10.3 告警建议 26
11. 故障排查速查 26
11.1 启动失败 27
11.2 接口异常 27
11.3 前端异常 28
11.4 数据异常 28
12. 升级与回滚 29
12.1 升级流程 29
12.2 回滚 29
附录 A：命令速查 30
附录 B：与上游文档的对应关系 30

0. 文档定位与阅读路径

你的角色	建议阅读顺序
运维实施（首次部署）	§1 → §2 → §3 → §4 → §5 → §11
运维实施（日常值守）	§6 → §7 → §8 → §10
开发（改代码后上线）	§3.4 → §9 → §12
安全/审计	§1.4 → §4.4 → §9.5
排障（出事了）	§11 故障速查表 → 对应小节

阅读约定

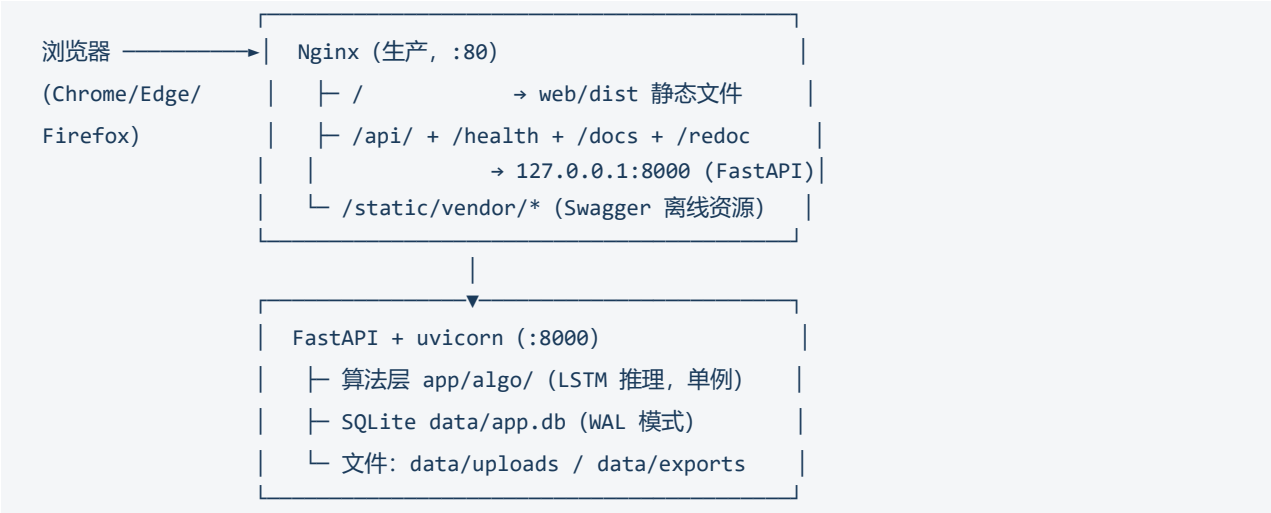
标记	含义
🔒	**硬约束。**写错会导致数据损坏、服务不可用或验收不通过，动手前必须确认
📌	**已确认决策。**本项目的既定选择，不要再改
⏱	本机实测数据（2026-10 在 MX450 / 8 核 / 24 GB 机器上实测）
⚠	易错点。都是本项目**真实踩过**的坑

---

1. 系统概览

1.1 架构

前后端分离的 B/S 架构，单机部署：



开发态（不经过 Nginx）由 Vite dev server（:5173）把 `/api`、`/health` 代理到后端 `:8000`。

1.2 组件与端口

组件	开发端口	生产端口	说明
前端（Vite dev）	5173	——	`strictPort: true`，5173 被占直接失败，不会静默换端口
前端（Nginx 静态）	——	80	托管 `web/dist/` 构建产物
后端（uvicorn）	8000	8000	`APP_PORT` 可覆盖
SQLite	——	——	文件型，无端口
`/docs` · `/redoc`	8000	8000	🔒 **资源已离线化**，见 §3.3

1.3 技术栈与版本（🔒 版本锁定，NFR-COMP-07）

层	选型	版本
语言	Python	3.12+（本机 3.12.14 / 3.13.12 实测可用）
框架	FastAPI / uvicorn	0.141.1 / 0.52.4

层	选型	版本
ORM / 驱动	SQLAlchemy	2.0.36
校验	Pydantic	2.9.2
认证	bcrypt / PyJWT	5.0.0 / 2.9.0
算法	torch	**2.9.1+cu128**（🔒 见 §2.3）
数据	pandas / numpy	2.3.3 / 2.5.3
导出	openpyxl	3.1.5
前端	React / Vite	18.3.1 / 5.4.9
UI	Ant Design / TailwindCSS	5.21.6 / 3.4.14
图表	ECharts	5.5.1（懒加载，不进主包）
数据库	SQLite	内置（WAL + 外键开启）

1.4 运行期硬约束（写进验收红线）

编号	约束	违反后果
🔒 `BE-05`	`JWT_SECRET` 缺失或 < 32 字符 → **启动直接失败**	若容错放行，任何人拿公开默认密钥即可伪造 Token
🔒 `BE-13`	每次连接必须 `PRAGMA foreign_keys = ON`	SQLite 默认关闭外键，删主记录不留孤儿、外键形同虚设
🔒 `BE-15`	启动时预加载模型，失败即启动失败	否则「服务正常、接口 200，但每次预测都用错版本」的静默故障
🔒 `GIT-7`	改算法参数后**必须**重跑训练与评估脚本	界面数字会落后于模型（本项目出现过「两套数字」）
📌 `QD-06`	首页硬指标从 `outputs/eval_report.json` 构建期注入	禁止手写，防「两套数字」复发
📌 训练时机	**纯预训练推理**，不提供在线重训	端到端 SLA 60 s 内无法稳定容纳「训练+调参+早停」

2. 环境准备

2.1 硬件最低配置

项	最低	推荐	说明
CPU	2 核	4 核+	无 GPU 时走 CPU 推理 ⏱ 实测 h=14 端到端 23.5 s
内存	4 GB	8 GB+	⏱ 实测峰值常驻 < 1 GB（模型仅 31,729 参数）
磁盘	2 GB 可用	10 GB+	模型 128 KB；数据库与上传文件按业务增长
GPU（可选）	——	支持 CUDA 的任意显卡	⏱ 显存占用峰值仅 **111 MB**，2 GB 的老卡也够

⚠ **别以为 LSTM 很吃显存。**本模型 hidden=48、2 层，参数量 31,729，⏱ 训练 100k 样本时峰值显存实测 111 MB。**1 分钟 SLA 之所以排除在线训练，理由是「训练+网格调参+早停无法稳定塞进 60 秒」，不是因为显存不够** —— 这个区别在答辩时会被追问。

2.2 系统依赖

```
# Linux (Debian/Ubuntu 系)
sudo apt-get update
sudo apt-get install -y python3.12 python3.12-venv python3-pip sqlite3 nginx curl


# Node.js 18+ (仅构建前端需要，运行期不需要)
node -v   # 建议 20.x / 22.x


# Windows：安装 Python 3.12+、Node.js 18+、以及任一现代浏览器
```

2.3 Python 依赖安装（⚠ torch 必须单独装）

```
python -m venv .venv
# Linux:  source .venv/bin/activate
# Windows: .venv\Scripts\activate


# ① 先装普通依赖
pip install -r requirements.txt


# ② 🔒 再单独装 CUDA 版 torch —— 直接走 PyPI 会装成 CPU 版
pip install torch==2.9.1+cu128 --index-url https://download.pytorch.org/whl/cu128
```

⚠ **国内网络**：`download.pytorch.org` 可能不可达。替代方案：

① 使用国内镜像（如阿里云 PyPI 镜像装 CPU 版 + 单独下 CUDA wheel）；

② 无 GPU 机器**直接装 CPU 版**即可 —— 算法层有设备自动回退（`FR-MOD-10`），

配置 `TORCH_DEVICE=cpu` 或 `auto` 都会正常工作，只是慢一些（⏱ 23.5 s vs 11.9 s）。

⚠ 安装完成后**必须验证**，别只看 `pip list`：

python -c "import torch; print(torch.__version__, torch.cuda.is_available())"

期望：2.9.1+cu128 True　（CPU 机器得到 2.9.1+cpu False 也是正常的）

2.4 必需的文件资产（🔒 缺一不可）

文件	位置	缺失后果
模型权重	`outputs/models/lstm_v1.pt`	🔒 启动失败（`BE-15`）
模型报告	`outputs/models/lstm_v1.report.json`	模型注册信息缺失，启动失败
示例数据集	`outputs/models/demo_sales.csv`	「一键加载示例数据」不可用
指标报告	`outputs/eval_report.json`	首页硬指标无法注入 → 前端构建失败
建库脚本	`code/db/schema.sql` + `seed.sql`	🔒 启动失败（建不出视图/触发器/游客哨兵行）
Swagger 资源	`app/static/vendor/swagger-ui/*`	`/docs` 回退 CDN → 国内网络下白屏

文件	位置	缺失后果
ReDoc 资源	`app/static/vendor/redoc/redoc.standalone.js`	`/redoc` 回退 CDN → 白屏

2.5 环境变量（.env）

复制 `.env.example` 为 `.env` 后按实际修改。**`.env` 已被 `.gitignore` 忽略，禁止提交。**

变量	默认	生产建议	说明
🔒 `JWT_SECRET`	空	**必填**，随机 ≥ 32 字符	缺失或过短 → 启动失败（`BE-05`）
`APP_ENV`	`development`	`production`	影响日志与错误细节暴露
`APP_PORT`	`8000`	`8000`	后端监听端口
`DB_PATH`	`data/app.db`	绝对路径	数据库文件
`UPLOAD_DIR`	`data/uploads`	同上	上传数据集落盘目录
`EXPORT_DIR`	`data/exports`	同上	导出产物目录（正式导出 7 天，临时明细 1 小时）
`MODEL_DIR`	`outputs/models`	同上	模型目录
`ACTIVE_MODEL`	`lstm_v1`	`lstm_v1`	🔒 必须与 `model_registry.is_active=1` 一致（`BE-06`）
`JWT_EXPIRE_MINUTES`	`720`	按需	Token 有效期（12 小时）
`LOGIN_MAX_FAIL`	`3`	`3`	连续失败几次锁定
`LOGIN_LOCK_SECONDS`	`60`	`60`	锁定时长
`MAX_FILE_SIZE_MB`	`20`	`20`	单文件上限
`MAX_ROWS`	`500000`	`500000`	单数据集行数上限
`MAX_SERIES`	`1000`	`1000`	单任务序列数上限
`TASK_BUDGET_MS`	`60000`	`60000`	🔒 端到端 SLA（硬约束，超时走软失败）
`EXPORT_TTL_DAYS`	`7`	`7`	正式导出产物保留天数
`EXPORT_TMP_TTL_SECONDS`	`3600`	`3600`	错误明细临时文件 TTL（1 小时）
`GUEST_DATA_TTL_HOURS`	`24`	`24`	游客数据优先清理窗口

变量	默认	生产建议	说明
CORS_ORIGINS	http://localhost:5173	生产填域名	逗号分隔；**生产态应由同源 Nginx 代理，此值可留空**
TORCH_DEVICE	auto	auto	auto / cpu / cuda

生成随机密钥：

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

---

3. 部署

3.1 目录结构

```
销量预测/
├── app/                    # 后端源码
│   ├── algo/               # 算法层（9 模块，可独立于 Web 层运行）
│   ├── api/v1/             # 路由层（auth/datasets/predictions/history/admin）
│   ├── core/               # 配置/DB/错误/日志/中间件/模型单例/安全
│   ├── dao/                # 数据访问层（8 个）
│   ├── models/             # ORM 实体（8 个）
│   ├── services/           # 业务服务层（8 个）
│   └── static/vendor/      # 🔒 Swagger / ReDoc 离线资源
├── web/                    # 前端工程（React + Vite）
│   ├── src/pages/          # 13 个页面
│   └── src/generated/      # 🔒 构建期注入产物（勿手改）
├── code/                   # 契约资产：db/schema.sql、seed.sql、schemas.py、types、api-client
├── data/                   # app.db / uploads / exports（运行期数据）
├── outputs/                # 模型、评估报告、交付文档
├── docs/                   # 全部文档源（Markdown）
├── scripts/                # 校验与运维脚本（35+ 套）
└── .env                    # 🔒 环境变量（不入库）
```

3.2 首次部署（开发/联调态，3 条命令）

```
# ① 后端（自动建库建表 + 建种子数据 + 预加载模型）
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000


# ② 前端 dev server（另开一个终端）
cd web && npm install && npm run dev


# ③ 验证
curl http://127.0.0.1:8000/health     # {"code":0,...,"data":{"status":"ok",...}}
# 浏览器打开 http://localhost:5173
```

⚠ 首次启动会执行 `init_db(run_seed=True)`：建表、建视图、建触发器、**补游客哨兵账号（`users.id = 0`）与管理员种子账号**。全过程幂等，重复启动无副作用。

⚠ 启动若报 `JWT_SECRET 未配置或长度不足 32 字符` —— 这是 `BE-05` 的**预期行为**，不是 bug。去 `.env` 里补一个 48 字符的随机串。

3.3 接口文档（/docs、/redoc）—— 🔒 资源已离线化

地址	内容
`/docs`	Swagger UI，可在线调试
`/redoc`	ReDoc，三栏阅读视图
`/openapi.json`	OpenAPI 3.1 契约（机器可读）
`outputs/电商销量预测系统_API 接口文档_V1.0.html`	**单文件离线归档件**，双击可开，零外链
`outputs/电商销量预测系统_openapi.json`	契约归档（可导入 Apifox / Postman）

为什么要自带资源：FastAPI 默认从 `cdn.jsdelivr.net` 加载 Swagger UI 的 JS/CSS。

国内受限网络下 CDN 被重置，症状是「HTML 拿到 200，但页面白屏、一个接口条目都渲染不出来」。

🔒 因此本项目把 `swagger-ui-bundle.js`、`swagger-ui.css`、favicon、`redoc.standalone.js` 随仓库自带（`app/static/vendor/`），并用自定义路由显式指向本地路径。

> ⚠ 两处**藏在默认参数里的外链**，只改主参数除不干净（EP-10 实测命中）：
>
> ① `get_redoc_html(..., redoc_favicon_url=...)` —— 默认 favicon 指向 `https://fastapi.tiangolo.com/img/favicon.png`；② `with_google_fonts=True` 会外链 Google Fonts。
>
> 本项目两者都已显式关闭/改写，并由 `scripts/verify_docs_browser.mjs` 逐条断言。

验证方式（🔒 必须用真实浏览器，`curl` 返 200 什么都不证明）：

```
node scripts/verify_docs_browser.mjs           # Edge
BROWSER=chrome node scripts/verify_docs_browser.mjs
```

三条判据缺一不可：① `.opblock` 数量 > 0（真的渲染了）；② DOM 内零外部 `http(s)` 引用；③ `Network.loadingFailed` 为空且无 4xx/5xx。当前状态：Edge / Chrome 各 **16/16 通过**。

3.4 生产构建与 Nginx

```
# ① 构建前端（tsc 类型检查 + vite build，产物在 web/dist/）
cd web
npm ci
VITE_API_TARGET=http://127.0.0.1:8000 npm run build     # 生产构建不需要该变量
cd ..


# ② 后端以生产模式启动（建议交给 systemd，见 §6.2）
APP_ENV=production python -m uvicorn app.main:app \
    --host 127.0.0.1 --port 8000 --workers 1
```

> 🔒 **`--workers` 必须是 1**。原因：登录限流的失败计数与黑名单是**进程内内存**实现（TQ-2 未引入 Redis）。多 worker 会让每个进程各持一份计数，限流形同虚设。

若将来确实需要多进程，**必须先把限流升级为共享存储**（Redis），这不是可以省略的前提。

Nginx 站点配置（`/etc/nginx/conf.d/sales-forecast.conf`）：

```
server {
    listen 80;
    server_name _;                      # 生产替换为真实域名
    client_max_body_size 25m;           # 🔒 必须大于 MAX_FILE_SIZE_MB(20)，留出表单开销

    # 前端静态资源（SPA 兜底）
    root /srv/sales-forecast/web/dist;
    index index.html;

    # 带 hash 的构建产物可长缓存
    location /assets/ {
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # 🔒 index.html 绝不缓存，否则发版后用户拿到旧壳、找不到新 chunk
    location = /index.html {
        add_header Cache-Control "no-store, must-revalidate";
    }

    # 后端 API
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 90s;         # 🔒 必须 > TASK_BUDGET_MS(60s)，否则长任务被 Nginx 掐断
    }

    location /health { proxy_pass http://127.0.0.1:8000; }

    # 🔒 接口文档与离线资源（Swagger/ReDoc 自带资源，不经 CDN）
    location /docs        { proxy_pass http://127.0.0.1:8000; }
    location /redoc       { proxy_pass http://127.0.0.1:8000; }
    location /openapi.json{ proxy_pass http://127.0.0.1:8000; }
    location /static/     { proxy_pass http://127.0.0.1:8000; }

    # SPA 兜底：非上述前缀一律回 index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;
    gzip_min_length 1024;
```

```
}
```

```
sudo nginx -t && sudo systemctl reload nginx
```

⚠ `proxy_read_timeout` 默认 60 s。若与 `TASK_BUDGET_MS=60000` 相等，边界上会来回打转：

⏱ 实测 3 任务并发时单个任务最长 37.6 s，仍在 60 s 内，但**极限规模压力用例（1000 序列 × horizon=30）后端要 250 s 才给出 `E_TIMEOUT`** —— 那种情况就是 Nginx 先掐断连接。

本项目的处理是**如实登记为「软失败路径」**（`P1-02`），生产若确有超大规模需求，

应把该接口改为异步 + 轮询，而不是把超时无限拉长。

3.5 部署后自检清单（🔒 逐项确认）

#	检查项	命令 / 方式	期望
1	健康检查	`curl /health`	`code: 0, status: ok`
2	密钥校验生效	临时清空 `JWT_SECRET` 启动	**启动失败**并报明确原因（`BE-05`）
3	外键已开启	看启动日志 / `PRAGMA foreign_keys`	启动时打印为 1（`BE-13`）
4	模型已预加载	启动日志	`模型已预加载 version=lstm_v1 device=...`
5	接口文档离线可用	`verify_docs_browser.mjs`	16/16
6	首页指标与算法一致	对比首页数字与 `outputs/eval_report.json`	逐项一致（`QD-06`）
7	游客链路可用	不登录直接访问上传页	可跑到结果页（`FR-USR-08`）
8	免责声明可见	任意页面页脚	111 字文案，不可折叠
9	管理员可登录	用种子账号登录	顶栏出现「管理后台」
10	上传大小限制	传 > 20 MB 文件	明确报错，不是 500

4. 配置详解

4.1 数据目录与 TTL 策略（三档）

目录	内容	TTL	清理入口
`data/uploads/`	上传的原始 CSV（按 UUID 落盘）	随数据集记录	删数据集时同步
`data/exports/`	正式导出产物（CSV）	**7 天**	`scripts/cleanup_tmp.py`
`data/exports/tmp/`	错误明细临时文件	**1 小时**	同上（`--with-tmp`）
游客数据（`user_id = 0`）	数据集 + 任务	**24 小时**	`cleanup_tmp.py --with-guest`

⚠ 正式导出与临时明细**必须分目录**（`exports/` 与 `exports/tmp/`）。混在一起时，7 天清理策略会误保留临时文件（本项目踩过，见实施方案「十八条坑」第 1 条）。

```
python scripts/cleanup_tmp.py --dry-run                  # 先看要删什么
python scripts/cleanup_tmp.py --with-tmp --with-guest    # 再真删
```

4.2 日志

日志	位置 / 方式	内容
访问日志	stdout（`AccessLogMiddleware`）	`[request_id] 方法 路径 状态码 耗时`
应用日志	stdout（`configure_logging`）	启动自检、模型加载、业务异常
systemd 日志	`journalctl -u sales-forecast`	生产态归集入口
Nginx 日志	`/var/log/nginx/access.log`、`error.log`	反代层

📌 每个请求都有唯一 `request_id`，且**贯穿访问日志与错误响应体**。排障时先拿到用户截图里的 `request_id`，再用 `journalctl -u sales-forecast | grep <request_id>` 定位，不要去猜时间点。

4.3 数据库参数（app/core/db.py，🔒 每次连接执行）

```
PRAGMA foreign_keys = ON;       -- 🔒 SQLite 默认 OFF，不开启则外键完全失效（BE-13）
PRAGMA journal_mode = WAL;      -- 一写多读并发（NFR-REL-04）
PRAGMA busy_timeout = 5000;     -- 写锁等待 5 秒而非立即报错
PRAGMA synchronous = NORMAL;    -- WAL 下的推荐值，兼顾安全与性能
```

⚠ **`PRAGMA journal_mode=WAL` 是「写一次、持久生效」的库级设置**；而 `foreign_keys` / `busy_timeout` 是**连接级**的，每条新连接都要重设 —— 这就是为什么它们写在连接工厂里，而不是建库时执行一次。

4.4 安全配置（生产必查）

项	建议	说明
`JWT_SECRET`	48 字符随机串，**仅存于服务端**	泄露 = 任何人可伪造任意用户 Token
`APP_ENV`	`production`	生产态不应把内部异常栈返回给前端
`CORS_ORIGINS`	生产**留空或填真实域名**	同源反代下浏览器不会发跨域请求
HTTPS	**必须**（6.2 数据隐私要求）	用 Nginx + 证书；`X-Forwarded-Proto` 已透传
限流	保持 `LOGIN_MAX_FAIL=3` / `LOCK=60s`	⚠ 该实现是进程内内存，**必须单 worker**
文件上传	`client_max_body_size` > `MAX_FILE_SIZE_MB`	两处不一致会出现「前端能选、Nginx 先 413」
目录权限	`data/`、`.env` 仅服务账号可读写	`chmod 600 .env`

5. 模型运维

5.1 模型资产

文件	说明
`outputs/models/lstm_v1.pt`	权重（128 KB）
`outputs/models/lstm_v1.report.json`	训练报告 + 超参 + 指标 + 逐轮 loss 历史
`outputs/eval_report.json`	端到端评估（四条硬指标 + 覆盖率 + 阶段耗时）
`outputs/eval_series.json`	逐序列预测结果（供前端原型/对比使用）

当前模型（`lstm_v1`）超参（取自 `lstm_v1.report.json`）：

项	值
隐藏层单元	48
层数	2
滑动窗口	21
特征数	11
参数量	**31,729**
训练样本 / 验证样本	8,140 / 850
最佳轮次	第 31 轮（早停于第 51 轮）
最佳验证损失	0.4741
训练耗时	⏱ 45.26 s（CUDA）

5.2 重训流程（🔒 GIT-7 纪律，顺序不可颠倒）

```
# ① 训练并注册（后台运行，前台易被中断）
python scripts/train_and_register.py


# ② 评估（必须 --export-series，且 horizon 与前端展示口径一致）
python scripts/eval_report.py --horizon 14 --export-series


# ③ 同步到前端（构建期注入，禁止手写）
python scripts/sync_eval_metrics.py        # → web/src/generated/eval_report.json
python scripts/sync_prototype_data.py      # → 原型数据（如启用）


# ④ 一致性自检（--check 只比对不写）
python scripts/sync_eval_metrics.py --check
python scripts/sync_pipeline_meta.py --check


# ⑤ 回归
python scripts/self_check.py               # 期望 69/69
```

⚠ **跳过 ③ 是本项目最典型的翻车方式**：模型变好了，但界面还显示旧数字，

于是「算法 14.07%、界面 14.26%」两套数字并存，答辩时无法解释。

`scripts/verify_ep3.py` 的硬指标同源断言就是为了拦住这种漂移。

5.3 模型切换（管理端）

管理端「模型版本」页（`AD-P01`）支持切换启用版本，实现要点：

1. 操作在一个事务内：`deactivate_all()` → `activate_version()`，**中间不得提前 commit**（`BE-ADM-02`）；
2. 切换后热重载：`reload_active_model()`，并校验 `.env` 的 `ACTIVE_MODEL` 与实际启用版本是否一致（`BE-ADM-03`）；
3. 不一致时返回 `warnings`，**不静默失败**；
4. 写审计：`operation_logs.action='admin'`，`detail` 形如 `sub=switch_model;target=lstm_v1;by=<admin>`，**与业务同事务**。

⚠ 手工改库改 `model_registry.is_active` 后**必须重启服务**：模型单例只在启动时加载。

更安全的做法是走管理端接口，它会自动重载。

---

6. 服务运行

6.1 开发态前台运行

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

6.2 生产态 systemd（推荐）

`/etc/systemd/system/sales-forecast.service`：

```
[Unit]
Description=电商销量预测系统 后端
After=network.target

[Service]
Type=simple
User=app
WorkingDirectory=/srv/sales-forecast
EnvironmentFile=/srv/sales-forecast/.env
Environment=APP_ENV=production
# 🔒 workers 固定 1：限流为进程内内存实现，多进程会使限流失效（见 §3.4）
ExecStart=/srv/sales-forecast/.venv/bin/python -m uvicorn app.main:app \
          --host 127.0.0.1 --port 8000 --workers 1
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now sales-forecast
sudo systemctl status sales-forecast
```

6.3 启动自检链（出问题优先看这里）

按顺序执行，任一失败即**启动失败**（不降级）：

```
① ensure_secret_configured()   → JWT_SECRET 长度 ≥ 32，否则 raise (BE-05)
② init_db(run_seed=True)       → 建表/视图/触发器 + PRAGMA + 种子（幂等）
③ load_active_model()          → 读 ACTIVE_MODEL 与 model_registry 一致性，
                                 加载权重、建计算图 (BE-15 / BE-06)
④ 打印「应用启动完成 env=... db=...」
```

在开发态用 `--reload` 时，**改任何 `app/` 下的代码都会重跑整条自检链**；

⚠ 若日志出现 `[Errno 10048] 端口被占用`（Windows）或 `Address already in use`，

说明上一个进程没退干净，先杀进程再重启（不要连点两次启动）。

---

7. 数据备份与恢复

7.1 备份（🔒 SQLite 必须用 .backup，不可直接 cp）

```
# ❌ 错误：WAL 模式下直接复制 .db，可能拿到缺 WAL 内容的不一致快照
# ✅ 正确：用 SQLite 的在线备份，自动处理 WAL
sqlite3 data/app.db ".backup 'backup/app-$(date +%F-%H%M).db'"

# 完整性校验（每次备份后都做）
sqlite3 backup/app-2026-10-13-0900.db "PRAGMA integrity_check;"   # 期望 ok
```

推荐：用仓库自带脚本（免手工、自带轮转与自检）

上面的手工步骤在实际运维里往往「不会被执行」，因此提供 `scripts/backup_db.py`：

```
python scripts/backup_db.py                 # data/app.db → backups/，保留 7 份
python scripts/backup_db.py --keep 30       # 保留 30 份
python scripts/backup_db.py --dry-run       # 只打印计划，不写盘
python scripts/backup_db.py --db data/app.db --out-dir /data/backup
```

它做四件事：① 用 `sqlite3.Connection.backup()` 在线复制（**WAL 下正确合并，无需停服务**）；② 按份数轮转；③ 对备份文件跑 `integrity_check` **并与源库逐表比对行数**（备份「存在」≠「可用」）；④ 输出一行摘要，便于重定向进日志。

定时执行：

```
# Linux cron：每天 03:00
0 3 * * * cd /opt/sales-forecast && python scripts/backup_db.py >> logs/backup.log 2>&1
```

Windows 用「任务计划程序」：程序 `python`，参数 `scripts/backup_db.py`，起始于项目根目录。

> ⚠ 两个实测坑（脚本已内置处理，手工做时要注意）：
> ① 备份库会继承源库的 `journal_mode=WAL`，读一次就生成 `-wal`/`-shm` 伴生文件，既污染备份目录又会让轮转漏删（脚本备份后切 `DELETE` 模式并清理伴生文件）；
> ② Windows 上「刚关闭连接立刻删」经常失败（句柄释放有延迟），轮转需带重试，否则日志说清理了、文件还在。

完整备份集应包含：

对象	备份必要性
`data/app.db`	🔒 必需（用户/数据集/任务/预测/指标/模型注册/审计/埋点）
`outputs/models/`	🔒 必需（模型权重 + 报告 + 示例数据）
`outputs/eval_report.json`	🔒 必需（首页指标来源）
`data/uploads/`	可选（原始 CSV 体量大，可用数据集元信息重建提示）
`.env`	🔒 必需（含 `JWT_SECRET`，丢失后所有已发 Token 失效）

7.2 恢复

```
sudo systemctl stop sales-forecast
cp backup/app-2026-10-13-0900.db data/app.db
rm -f data/app.db-wal data/app.db-shm        # 🔒 必须删掉旧 WAL/SHM，否则与恢复的库不匹配
python -c "import sqlite3;c=sqlite3.connect('data/app.db');print(c.execute('PRAGMA
integrity_check').fetchone())"
sudo systemctl start sales-forecast
curl -s http://127.0.0.1:8000/health
```

⚠ 恢复数据库后**必须验证模型一致性**：若备份里的 `model_registry` 启用版本与当前 `outputs/models/` 里的权重不匹配，启动会因 `BE-06` 失败。这是**保护**，不是故障。

7.3 数据库迁移

📌 本项目**不引入 Alembic**（`TD-DB-03`），用 `PRAGMA user_version` 手工管理：

```
python scripts/migrate_0001_ch01_role.py    # 幂等 + 迁移前自动备份
sqlite3 data/app.db "PRAGMA user_version;"  # 期望 1
python scripts/verify_schema.py             # 对象齐备性
python scripts/verify_orm.py                # ORM ⇔ DDL 一致性
```

8. 定期运维任务

周期	任务	命令
每日	导出与临时文件清理	`python scripts/cleanup_tmp.py --with-tmp --with-guest`
每日	健康检查	`curl -s /health`
每周	数据库完整性 + 备份	`sqlite3 data/app.db "PRAGMA integrity_check;" + .backup`
每周	失败任务与错误码巡检	`sqlite3 data/app.db "select error_code,count(*) from tasks where status='failed' group by 1;"`
每月	审计日志巡检	`select action,sub,count(*) from operation_logs group by 1,2;`
每月	磁盘水位	`du -sh data/ outputs/`
每季度	全量回归	见 §9

> 💡 三个 PRAGMA 是排障三件套：
>
> `integrity_check`（页面损坏）、`foreign_key_check`（孤儿行）、`user_version`（结构版本）。

9. 质量保障与回归

9.1 校验脚本矩阵（🔒 全部为可复现断言，非人工目视）

类别	脚本	覆盖
算法	`self_check.py`	算法层自检 69 项

类别	脚本	覆盖
	eval_report.py	端到端评估 + 四条硬指标
数据/模型	verify_orm.py / verify_schema.py / verify_schemas.py	ORM↔DDL、对象齐备、Pydantic
前端静态	verify_types.mjs / verify_tokens.py / verify_prototype.mjs	类型、设计令牌、原型回归
文档	verify_doc_refs.py / verify_doc_sql.py	编号交叉引用、SQL 一致性
认证	verify_auth.py	Token 生命周期 / 越权 / 改密撤回
分阶段静态	verify_ep3.py … verify_ep8.py	各阶段实现约束（含 AST 红线扫描）
分阶段接口	verify_ep4_api.py … verify_ep8_api.py、verify_ep5_equiv.py	各阶段端到端
浏览器走查	verify_ep3_browser.mjs … verify_ep8_browser.mjs	真实浏览器逐页断言
EP-9	verify_ep9_api.py / _perf.py / _browser.mjs / _contract.py / verify_ep9.py	边界 / 性能 / 兼容 / 契约 / 清单守卫
EP-10	verify_ep10_docs.py / verify_docs_browser.mjs	交付物齐备 / 接口文档离线化
无障碍	a11y_audit.py	53 项，退出码 0
运维	cleanup_tmp.py	三档 TTL 清理

9.2 回归执行纪律（⚠ 血泪教训）

每个校验脚本单独占一条命令执行，不要用 `for` 循环串跑。

原因：平台的批量删除保护按**单次调用的累计删除数**计数（阈值 50）。

多个脚本各自清理自己的临时目录，串跑会让删除量累加到同一次调用上 → 被判为批量删除 →

后续脚本直接退出，日志只留一行 `STATUS: FAILED`。EP-9 首轮就因此出现三个脚本「假失败」。

9.3 发布前必跑

```
python scripts/verify_doc_refs.py        # 0 悬空编号
python scripts/verify_schema.py
python scripts/verify_orm.py
python scripts/self_check.py             # 69/69
python scripts/sync_eval_metrics.py --check
python scripts/sync_pipeline_meta.py --check
python scripts/verify_ep10_docs.py
node scripts/verify_docs_browser.mjs
```

---

10. 监控与容量

10.1 关键指标（⏱ 本机实测基线，2026-10-12）

指标	实测	SLA / 期望	告警阈值建议
端到端（h=14，113 序列）	**11.85 s**	≤ 60 s（`TASK_BUDGET_MS`）	> 30 s
其中推理阶段	21 ms	——	——
预处理 / 特征 / 指标阶段	675 / 763 / 473 ms	——	单阶段 > 5 s
CPU 回退（无 GPU）	23.49 s	≤ 60 s	> 40 s
数据集预览接口	55.7 ms	≤ 1 s	> 500 ms
历史列表首屏	61.4 ms	≤ 1 s	> 500 ms
首屏 FCP（生产构建）	504 ms（Edge）/ 140 ms（Chrome）	≤ 3 s	> 2 s
3 任务并发（各自）	32.2 / 37.6 / 37.5 s	全部成功、锁冲突 0	任一 > 50 s
极限规模（1000 序列 × h=30）	250.3 s → `E_TIMEOUT`	**软失败**：无 5xx、无半成品	出现 5xx 即告警
数据库行数	8 表 / 121 列 / 1 视图 / 3 触发器 / 12 索引	——	`predictions` > 100 万行需重评（`TD-DB-01`）

10.2 容量估算

维度	单条记录规模	说明
预测明细	约 60–120 字节/行	113 序列 × 14 天 ≈ 1,582 行 ≈ 180 KB
数据集	约 40 字节/行	500k 行上限 ≈ 20 MB（与 `MAX_FILE_SIZE_MB=20` 对齐）
磁盘增长	取决于任务量	`predictions` 是主要增长源；超过 100 万行需重新评估索引与分页

10.3 告警建议

事件	判据	动作
服务不可用	`/health` 连续 3 次失败	重启 + 看启动自检日志
任务大面积失败	`tasks` 中 `status='failed'` 占比 > 20%	查 `error_code` 分布，优先看 `E_TIMEOUT` / `40404`
`40404`（模型未加载）	出现即告警	说明模型单例被清或启动失败，**不是用户操作问题**
磁盘水位	`data/` + `outputs/` > 80%	跑 `cleanup_tmp.py`，再评估归档
限流频繁触发	同 IP 频繁 `42901`	可能是撞库，检查审计日志

---

11. 故障排查速查

11.1 启动失败

现象	根因	处置	
`JWT_SECRET` 未配置或长度不足 32 字符	`BE-05` 预期保护	在 `.env` 补 ≥ 32 字符随机串	
`model_registry` 中没有启用记录	库没建好或种子未跑	确认 `init_db` 成功；或 `ACTIVE_MODEL` 与库不一致	
配置漂移：`.env` 与库中启用版本不一致	`BE-06` 预期保护	把 `ACTIVE_MODEL` 改成库里的启用版本（或反之）	
`Address already in use` / `[Errno 10048]`	上一个进程未退	按端口找 PID 杀掉再启（Windows：`netstat -ano \	grep :8000`）
`no such table: v_task_summary`	用了 `create_all` 而非 `schema.sql`	🔒 必须执行 `code/db/schema.sql`（视图/触发器/哨兵行只在那里）	

11.2 接口异常

现象	根因	处置
所有预测接口 `40404`	模型单例未加载	看启动日志；`TestClient` 场景还须**全程同一 lifespan**
上传接口报错但库里没记录	状态变更后 `raise BizError`，被 `get_db` 回滚	🔒 需要保留的副作用**在抛错前显式 `db.commit()`**
登录限流永不生效	用哨兵时间（`datetime.min`/`0`）表示「未锁定」	🔒 「计数中」用 `None`、「已锁定」用 `datetime`，必须不同类型
历史对比把 `compare` 当 `task_id`	路由注册顺序错误	🔒 `POST /history/compare` 必须排在 `/{task_id}` 之前（`BE-39`）
非管理员访问管理端返回 403 而非 401	两种拒绝混用	未登录 → `401`；已登录无权限 → `40301`

11.3 前端异常

现象	根因	处置
页面能开但**每个接口都失败**、后端日志无请求	Vite 代理指向错误端口	设 `VITE_API_TARGET`（默认写死 8000）
页面全白、控制台无报错	浏览器继承了系统代理，本地地址被塞进代理 → 502	浏览器加 `--no-proxy-server`；或清掉 `HTTP_PROXY`
`/docs` 白屏但返回 200	Swagger 资源走 CDN 被墙	🔒 确认 `app/static/vendor/swagger-ui/` 资源存在
`/redoc` 控制台出现 `fastapi.tiangolo.com` 请求	默认 favicon 未改写	传 `redoc_favicon_url`（⚠ 不是 `favicon_url`）
首屏偶发取不到 FCP 样本	`goto` 后 settle 太短，React 还没画	读取加重试，**不要**放宽断言为 `n ≥ 2`
断网环境下页面空白	前端引用了外部字体/图标	本项目前端**零外链**；若新增依赖须复查

11.4 数据异常

现象	根因	处置
删除主记录后出现孤儿行	外键未开启	🔒 每次连接 `PRAGMA foreign_keys = ON`（BE-13）
恢复备份后行为怪异	旧 WAL/SHM 残留	恢复时同步删除 `app.db-wal` / `app.db-shm`
指标显示与算法不一致	忘了跑 `sync_eval_metrics.py`	🔒 GIT-7：重训后必须重跑注入脚本
数据库文件损坏	直接 `cp` 备份导致快照不一致	改用 `sqlite3 .backup`

12. 升级与回滚

12.1 升级流程

```
# ① 备份（🔒 不可省）
sqlite3 data/app.db ".backup 'backup/pre-upgrade-$(date +%F-%H%M).db'"
tar czf backup/pre-upgrade-models-$(date +%F).tgz outputs/models outputs/eval_report.json


# ② 停服
sudo systemctl stop sales-forecast


# ③ 取新代码 + 装依赖
git pull
pip install -r requirements.txt
cd web && npm ci && npm run build && cd ..


# ④ 迁移（若有 schema 变更）
python scripts/migrate_0001_ch01_role.py


# ⑤ 回归
python scripts/self_check.py
python scripts/verify_ep10_docs.py


# ⑥ 起服 + 自检
sudo systemctl start sales-forecast
curl -s http://127.0.0.1:8000/health
```

12.2 回滚

```
sudo systemctl stop sales-forecast
git checkout <上一个正常版本 tag>
cp backup/pre-upgrade-<时间>.db data/app.db
rm -f data/app.db-wal data/app.db-shm
tar xzf backup/pre-upgrade-models-<日期>.tgz
cd web && npm ci && npm run build && cd ..
sudo systemctl start sales-forecast
```

⚠ 回滚数据库是**破坏性操作**：升级后产生的所有业务数据都会丢失。

仅在升级后确认数据不可用时执行，且执行前**再备份一次当前库**留证。

附录 A：命令速查

```
# ---- 启动 / 停止 ----
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000        # 开发态
sudo systemctl start|stop|restart|status sales-forecast            # 生产态
journalctl -u sales-forecast -f | grep <request_id>                # 按请求追踪


# ---- 健康与自检 ----
curl -s http://127.0.0.1:8000/health
python scripts/self_check.py
python scripts/verify_ep10_docs.py


# ---- 备份 / 恢复 ----
sqlite3 data/app.db ".backup 'backup/app-$(date +%F-%H%M).db'"
sqlite3 data/app.db "PRAGMA integrity_check;"
sqlite3 data/app.db "PRAGMA foreign_key_check;"
sqlite3 data/app.db "PRAGMA user_version;"


# ---- 清理 ----
python scripts/cleanup_tmp.py --dry-run
python scripts/cleanup_tmp.py --with-tmp --with-guest


# ---- 模型 ----
python scripts/train_and_register.py
python scripts/eval_report.py --horizon 14 --export-series
python scripts/sync_eval_metrics.py
python scripts/sync_eval_metrics.py --check


# ---- 接口文档 ----
python scripts/export_openapi.py
python scripts/build_api_doc_html.py
node scripts/verify_docs_browser.mjs
```

附录 B：与上游文档的对应关系

本文档小节	上游依据
§1.3 技术栈	《项目任务书》§3.2、《开发技术文档》§1.2
§1.4 硬约束	《开发技术文档》§4（`BE-*` 编号）
§2.5 环境变量	《开发技术文档》§2.4
§3.3 接口文档离线化	《开发技术文档》§10.4
§3.4 Nginx	《开发技术文档》§10（部署章节）
§4.1 TTL 三档	《数据库设计文档》（`JT-10`/`JT-11`/`DB-12`）
§9 校验矩阵	《项目开发实施方案》§9.1、《测试报告》§2.11
§10 性能基线	《开发技术文档》§9.7、附录 D

文档结束。本手册与《开发技术文档 V1.0》冲突时，**技术口径以技术文档为准**；

与《项目开发实施方案 V1.0》冲突时，**流程口径以实施方案为准。**